

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Методы трансляции»

ОТЧЕТ
к лабораторной работе № 5
на тему «Интерпретация исходного кода»

Выполнил

Е. С. Кахновский

Проверил

Н. Ю. Гриценко

Минск 2024

СОДЕРЖАНИЕ

1 Постановка задачи.....	3
2 Краткие теоретические сведения.....	4
3 Результаты выполнения лабораторной работы.....	5
Выводы	6
Список использованных источников	7
Приложение А (обязательное) Листинг исходного кода	8

1 ПОСТАНОВКА ЗАДАЧИ

Целью выполнения данной лабораторной работы является на основе результатов анализа лабораторных работы 1-4 выполнить трансляцию программы с языка программирования C++ на язык программирования Python, после чего выполнить интерпретацию программы.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

К этапам трансляции относятся следующие этапы:

- лексический анализ;
- синтаксический анализ;
- семантический анализ;
- оптимизация;
- генерация кода.

На этапе генерации компилятор создает код, который представляет собой набор инструкций, понятных для целевой аппаратной платформы, итоговый файл компилируется в исполняемый файл, который может быть запущен на целевой платформе без необходимости наличия кода.

Фаза эмуляции интерпретатора происходит во время выполнения программы. В отличие от компилятора, интерпретатор работает с кодом напрямую, без предварительной генерации машинного кода.

Лексический анализатор – первый этап трансляции. Лексический анализатор читает поток символов, составляющих исходную программу, и группирует эти символы в лексемы или значащие последовательности. Лексема – это элементарная единица, которая может являться ключевым словом, идентификатором, константным значением. Для каждой лексемы анализатор строит токен, который по сути является кортежем, содержащим имя и значение.[1]

Синтаксический анализатор выясняет, удовлетворяют ли предложения, из которых состоит исходная программа, правилам грамматики языка программирования. Синтаксический анализатор получает на вход результат лексического анализатора и разбирает его в соответствии с грамматикой. Результат синтаксического анализа обычно представляется в виде синтаксического дерева разбора.[2]

Семантический анализ обычно заключается в проверке правильности типа и вида всех идентификаторов и данных, используемых в программе.

Семантический анализатор использует синтаксическое дерево и информацию из таблицы символов для проверки исходной программы на семантическую согласованность с определением языка. Он также собирает информацию о типах и сохраняет ее в синтаксическом дереве или в таблице идентификаторов для последующего использования в процессе генерации промежуточного кода.

В данной лабораторной работе были использованы результаты анализа лексического, синтаксического и семантического анализаторов, после чего каждый узел дерева разбора был переведен с языка программирования C++ на язык программирования Python. После чего была выполнена интерпретация программ. Программами называются тестовые исходные коды, представленные в лабораторной работе 1.

3 РЕЗУЛЬТАТЫ ВЫПОЛНЕНИЯ ЛАБОРАТОРНОЙ РАБОТЫ

В ходе лабораторной работы был реализован транслятор программ с языка программирования C++ на язык программирования Python с последующей интерпретацией кода.

Листинг первого тестового кода представлен на рисунке 3.1.

```
#include <iostream>

int main() {
    int n = 5;
    int arr[] = {64, 25, 12, 22, 11};
    for (int i = 0; i < n - 1; ++i) {
        for (int j = 0; j < n - i - 1; ++j) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
    std::cout << "Sorted array: " << std::endl;
    for (int i = 0; i < n; ++i) {
        std::cout << arr[i] << std::endl;
    }
    std::cout << std::endl;
    return 0;
}
```

Рисунок 3.1 – Листинг первого тестового кода

Результат трансляции первого тестового кода представлен на рисунке 3.2.

```
if __name__ == "__main__":
    n = 5
    arr = [ 64,      25,      12,      22,      11]
    for i in range(    n - 1):
        for j in range(        n -      i - 1):
            if      arr[          j]>      arr[          j + 1]:
                temp =      arr[          j]
                arr[          j] =      arr[          j + 1]
                arr[          j + 1] =      temp

    print( "Sorted array: ")
    for i in range(        n):
        print(      arr[      i])

    print()
```

Рисунок 3.2 – Результат трансляции первого исходного кода

ВЫВОДЫ

В ходе лабораторной работы был реализован интерпретатор для программ на языке C++, который переводит их на язык программирования Python после чего проводит интерпретацию полученного при трансляции кода.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Лексический анализатор [Электронный ресурс]. – Режим доступа: <https://csc.sibsutis.ru/sites/csc.sibsutis.ru/files/courses/trans/>. – Дата доступа: 27.02.2024.

[2] Синтаксический анализатор [Электронный ресурс]. – Режим доступа: <https://csc.sibsutis.ru/sites/csc.sibsutis.ru/files/courses/trans/>. – Дата доступа: 27.02.2024.

[3] Введение в C++ [Электронный ресурс]. – Режим доступа: <https://metanit.com/cpp/tutorial/2.5.php>. – Дата доступа: 28.02.2024.

[4] Типы данных [Электронный ресурс]. – Режим доступа: <https://metanit.com/cpp/tutorial/2.3.php>. – Дата доступа: 28.02.2024.

[5] Операторы в C++ [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/ru-ru/cpp/c-operators>. – Дата доступа: 27.02.2024.

[6] Функции C++ [Электронный ресурс]. – Режим доступа: <https://metanit.com/cpp/tutorial/3.1.php>. – Дата доступа: 27.02.2024.

[7] Классы C++ [Электронный ресурс]. – Режим доступа: <https://ravesli.com/urok-113-klassy-obekty-i-metody-klassov/>. – Дата доступа: 27.02.2024.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг исходного кода

Листинг 1 – Программный код parser.py

```
from parser import parser
class Translator:
    def __init__(self):
        self.code_python = ''
        self.private_member = []
        self.variable_list = []
        self.temp_list = ''
        self.in_class = False
        self.in_if = False
        self.depth = 0
        self.temp_depth = 0
        self.count_compare = 0

    def declare_translator(self, node):
        self.code_python += '\t' * self.depth
        variable_exists = False
        for name, _ in self.variable_list:
            if node.name == name:
                variable_exists = True
                break

        if not variable_exists:
            if self.in_class:
                self.variable_list.append((node.name, True))
            else:
                self.variable_list.append((node.name, False))

        for name, check in self.variable_list:
            if node.name == name:
                if check:
                    self.code_python += f'self.{node.name}'
                else:
                    self.code_python += f'{node.name}'
                break

        for member_declare in node.children:
            if member_declare.type == 'Assignment':
                assignment_node = member_declare.name
                self.code_python += f' {assignment_node} '
            if member_declare.type == 'Value':
                value_node = member_declare.name
                self.code_python += f' {value_node}'
            if member_declare.type == 'Variable':
                self.declare_translator(member_declare)
            if member_declare.type == 'Operator':
                operator_node = member_declare.name
                self.code_python += f' {operator_node}'
```